PMSCS 670

Software Testing

Testing Fundamentals

Lecture 2

Mohammad Ashraful Islam

What is Software?

- Software is a program that enables a computer to perform a specific task.
- This includes:
  - application software such as a word processor, which enables a user to perform a task,
  - system software such as an operating system, which enables other software to run properly, by interfacing with hardware and with other software.
- Practical computer systems divide software into three major classes:
  - system software,
  - programming software and
  - application software,

  although the distinction is arbitrary, and often blurred.

What is Software Testing?

- **Software testing** is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

- **According to ANSI/IEEE 1059 standard** – Software testing is a process of analyzing a software item to detect the differences between existing and required conditions (i.e., defects) and to evaluate the features of the software item.

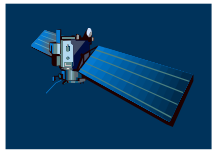
Software in the 21st Century

- Software defines behavior
  - network routers, finance, switching networks, other infrastructure
- Today's software market :
  - is much bigger
  - is more competitive
  - has more users
- Automated/ Embedded Control Applications
  - airplanes, air traffic control
  - spaceships
  - watches
  - ovens
  - remote controllers
  - memory seats
  - PDAs
  - DVD players
  - garage door openers
  - cell phones

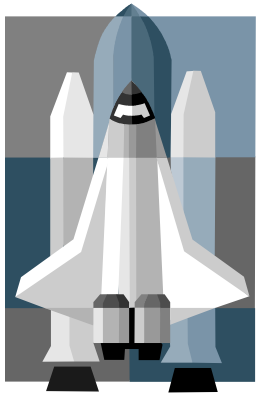
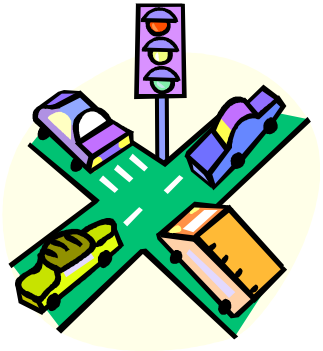
Software is a Skin that Surrounds Our Civilization









Quote due to Dr. Mark Harman

Quality Software

- **Quality software** refers to a software which is reasonably bug or defect free, is delivered in time and within the specified budget, meets the requirements and/or expectations, and is maintainable.
- In the software engineering context, software quality reflects both **functional quality** as well as **structural quality**.
  - **Software Functional Quality** − It reflects how well it satisfies a given design, based on the functional requirements or specifications.
  - **Software Structural Quality** − It deals with the handling of non-functional requirements that support the delivery of the functional requirements, such as robustness or maintainability, and the degree to which the software was produced correctly.

Quality Software Cont...

- **Software Quality Assurance** − Software Quality Assurance (SQA) is a set of activities to ensure the quality in software engineering processes that ultimately result in quality software products. The activities establish and evaluate the processes that produce products. It involves process-focused action.

- **Software Quality Control** − Software Quality Control (SQC) is a set of activities to ensure the quality in software products. These activities focus on determining the defects in the actual products produced. It involves product-focused action.

The Software Quality Challenges

- In the software industry, the developers will never declare that the software is free of defects, unlike other industrial product manufacturers usually do.
- This difference is due to the following reasons.
  - **Product Complexity** - It is the number of operational modes the product permits. Normally, an industrial product allows only less than a few thousand modes of operation with different combinations of its machine settings. However, software packages allow millions of operational possibilities. Hence, assuring of all these operational possibilities correctly is a major challenge to the software industry.
  - **Product Visibility** - Since the industrial products are visible, most of its defects can be detected during the manufacturing process. Also the absence of a part in an industrial product can be easily detected in the product but, the defects in software products which are stored on diskettes or CDs are invisible.

Software vs Other Products

- Comparison **software products** vs **industrial products**:

Characteristic	Software Products	Other Industrial Products
Complexity	Millions of operational options	Thousand operational options
Visibility of product	Invisible Product. Difficult to detect defects by sight	Visible Product. Effective detection of defects by sight
Nature of development and production process	Can detect defects in only development phase	Can detect defects in all of the following phases •Product development •Product production planning •Manufacturing

How we will get quality software?

Software Faults, Errors & Failures

- Software Fault : A static defect in the software
- Software Error : An incorrect internal state that is the manifestation of some fault
- Software Failure : External, incorrect behavior with respect to the requirements or other description of the expected behavior

Faults in software are equivalent to design mistakes in hardware.

Software does not degrade.

Fault and Failure Example

- A patient gives a doctor a list of symptoms
  - Failures
- The doctor tries to diagnose the root cause of the ailment
  - Fault
- The doctor may look for anomalous internal conditions (high blood pressure, irregular heartbeat, bacteria in the blood stream)
  - Errors

Most medical problems result from external attacks (bacteria, viruses) or physical degradation as we age.

Software faults were there at the beginning and do not "appear" when a part wears out.

A Concrete Example

```
public static int numZero (int [ ] arr)
{   // Effects: If arr is null throw NullPo          on
    // else return the number of occurrence          r
    int count = 0;
    for (int i = 1; i < arr.length; i++)
    {
        if (arr [ i ] ==
        {
            count++;
        }
    }
    return count;
}
```

Fault: Should start searching at 0, not 1

Test 1
[2, 7, 0]
Expected: 1
Actual: 1

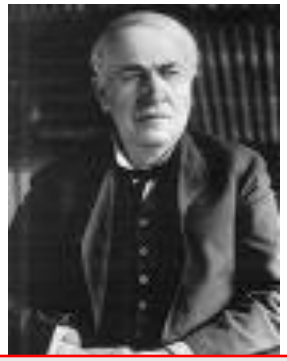
Error: i is 1, not 0, on the first iteration
Failure: none

Test 2
[0, 2, 7]
Expected: 1
Actual: 0

Error: i is 1, not 0
Error propagates to the variable count
Failure: count is 0 at the return statement

The Term Bug

- *Bug* is used informally
- Sometimes speakers mean fault, sometimes error, sometimes failure … often the speaker doesn't know what it means !
- This class will try to use words that have precise, defined, and unambiguous meanings



"It has been just so in all of my inventions. The first step is an intuition, and comes with a burst, then difficulties arise—this thing gives out and *[it is]* then that 'Bugs'—as such little faults and difficulties are called—show themselves and months of intense watching, study and labor are requisite. . ." – Thomas Edison

"an analyzing process must equally have been performed in order to furnish the Analytical Engine with the necessary operative data; and that herein may also lie a possible source of error. Granted that the actual mechanism is unerring in its processes, the cards may give it wrong orders. " – Ada, Countess Lovelace (notes on Babbage's Analytical Engine)

What is a computer bug?

- In 1947 Harvard University was operating a room-sized computer called the Mark II.
  - mechanical relays
  - glowing vacuum tubes
  - technicians program the computer by reconfiguring it
  - Technicians had to change the occasional vacuum tube.

- A moth flew into the computer and was zapped by the high voltage when it landed on a relay.

- Hence, the first computer bug!
  - I am not making this up :-)

Bugs a.k.a. ...

- Defect
- Fault
- Problem
- Error
- Incident
- Anomaly
- Variance
- Failure
- Inconsistency
- Product Anomaly
- Product Incidence
- Feature :-)

We will use the term in proper way.

Defective Software

- We develop programs that contain defects
  - How many? What kind?
- Hard to predict the future, however…

  it is highly likely, that the software we (including you!) will develop in the future will not be significantly better.

Sources of Problems

- **Requirements Definition:** Erroneous, incomplete, inconsistent requirements.
- **Design:** Fundamental design flaws in the software.
- **Implementation:** Mistakes in chip fabrication, wiring, programming faults, malicious code.
- **Support Systems:** Poor programming languages, faulty compilers and debuggers, misleading development tools.
- **Inadequate Testing of Software:** Incomplete testing, poor verification, mistakes in debugging.
- **Evolution:** Sloppy redevelopment or maintenance, introduction of new flaws in attempts to fix old flaws, incremental escalation to inordinate complexity.

Adverse Effects of Faulty S/W

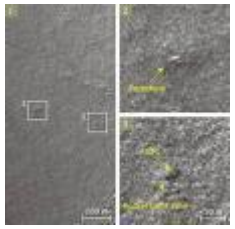
- **Communications:** Loss or corruption of communication media, non-delivery of data.
- **Space Applications:** Lost lives, launch delays.
- **Defense and Warfare:** Misidentification of friend or foe.
- **Transportation:** Deaths, delays, sudden acceleration, inability to brake.
- **Safety-critical Applications:** Death, injuries.
- **Electric Power:** Death, injuries, power outages, long-term health hazards (radiation).

Adverse Effects of Faulty S/W Cont

- **Money Management:** Fraud, violation of privacy, shutdown of stock exchanges and banks, negative interest rates.
- **Control of Elections:** Wrong results (intentional or non-intentional).
- **Control of Jails:** Technology-aided escape attempts and successes, accidental release of inmates, failures in software controlled locks.
- **Law Enforcement:** False arrests and imprisonments.

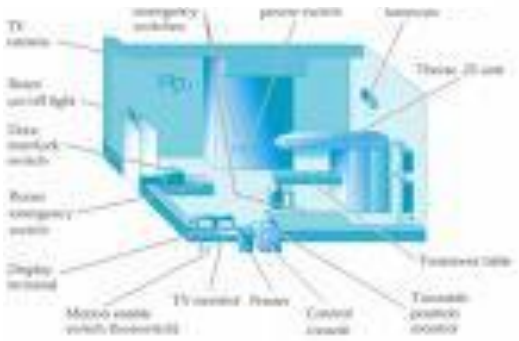
Spectacular Software Failures

- NASA's Mars lander: September 1999, crashed due to a units integration fault



Mars Polar Lander crash site?

THERAC-25 design



- THERAC-25 radiation machine : Poor testing of safety-critical software can cost *lives* : 3 patients were killed
- Ariane 5 explosion : Millions of $$
- Intel's Pentium FDIV fault : Public relations nightmare

Ariane 5: exception-handling bug : forced self destruct on maiden flight (64-bit to 16-bit conversion: about 370 million $ lost)

We need our software to be dependable.
***Testing* is *one* way to assess dependability.**

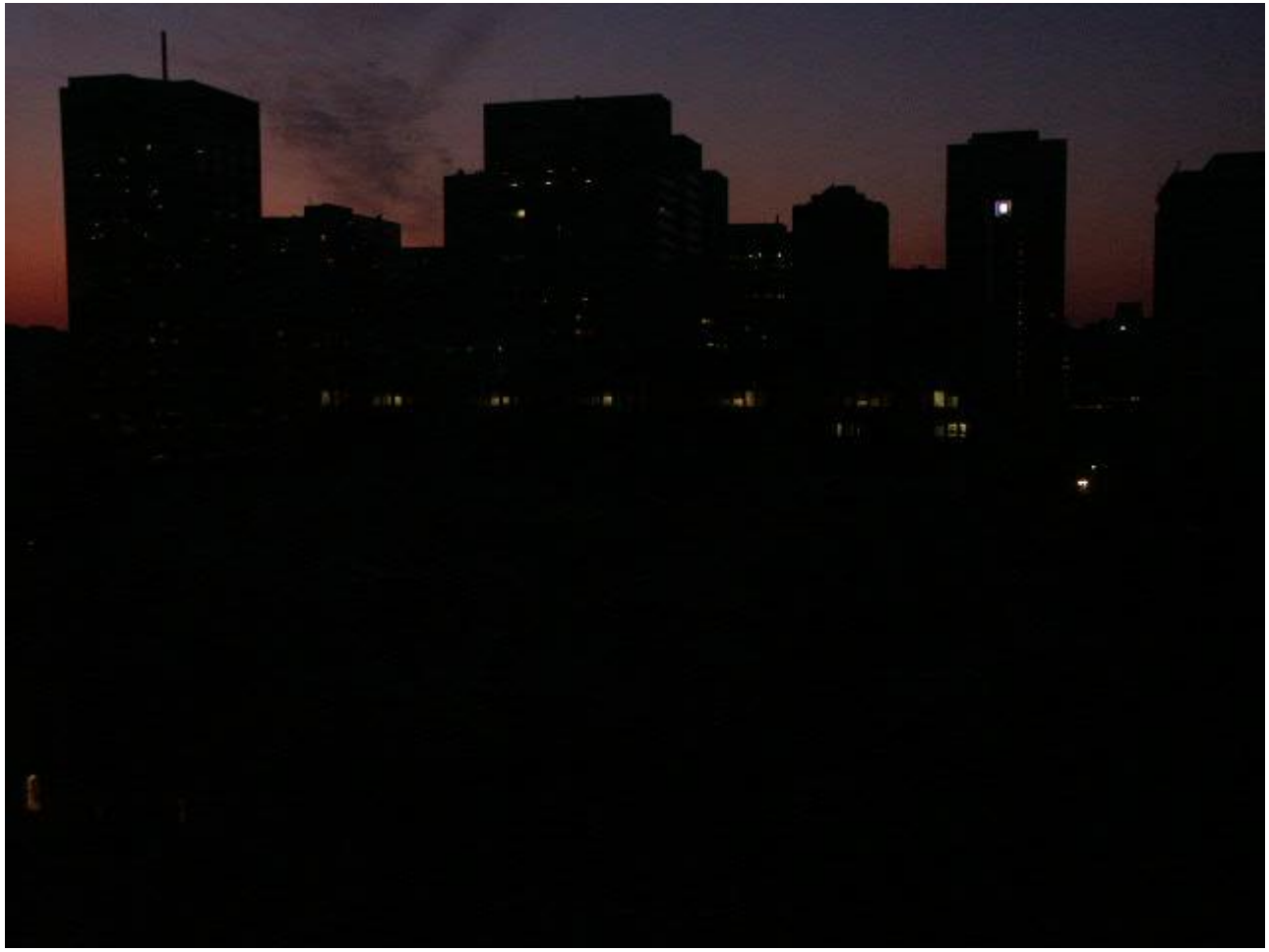
Northeast Blackout of 2003

508 generating units and 256 power plants shut down

Affected 10 million people in Ontario, Canada

Affected 40 million people in 8 US states

Financial losses of $6 Billion USD



The alarm system in the energy management system failed due to a software error and operators were not informed of the power overload in the system

Bank Generosity

- A Norwegian bank ATM consistently dispersed 10 times the amount required.
- Many people joyously joined the queues as the word spread.

Bank Generosity (Cont'd)

- A software flaw caused a UK bank to duplicate every transfer payment request for half an hour.
- The bank lost 2 billion British pounds!
- The bank eventually recovered the funds but lost half a million pounds in potential interest.

Making Rupee!

- An Australian man purchased $104,500 worth of Sri Lankan Rupees.
- The next day he sold the Rupees to another bank for $440,258.
- The first bank's software had displayed a bogus exchange rate in the Rupee position!
- A judge ruled that the man had acted without intended fraud and could keep the extra $335,758!

Bug in BoNY Software

- The Bank of New York (BoNY) had a $32 billion overdraft as the result of a 16-bit integer counter that went unchecked.
- BoNY was unable to process the incoming credits from security transfers, while the NY Federal Reserve automatically debited BoNY's cash account.
- BoNY had to borrow $24 billion to cover itself for one day until the software was fixed.
- The bug cost BoNY $5 million in interest payments.

Costly Software Failures

- NIST report, "The Economic Impacts of Inadequate Infrastructure for Software Testing"
  - Inadequate software testing costs the US alone between $22 and $59 billion annually
  - Better approaches could cut this amount in half
- Huge losses due to web application failures
  - Financial services : $6.5 million per hour (just in USA!)
  - Credit card sales applications : $2.4 million per hour (in USA)
- In Dec 2006, *amazon.com's* BOGO offer turned into a double discount
- 2007 : Symantec says that most security vulnerabilities are due to faulty software

World-wide monetary loss due to poor software is shocking

Testing in the 21st Century

- More safety critical, real-time software
- Embedded software is ubiquitous … check your pockets
- Enterprise applications means bigger programs, more users
- Paradoxically, free software increases our expectations !
- Security is now all about software faults
  - Secure software is reliable software
- The web offers a new deployment platform
  - Very competitive and very available to more users
  - Web apps are distributed
  - Web apps must be highly reliable

Industry desperately needs our inventions !

What Does This Mean?

Software testing is getting more important

What are we trying to do when we test ?

What are our goals ?

Verification & Validation (*IEEE*)

- Verification : The process of determining whether the products of a given phase of the software development process fulfill the requirements established during the previous phase

- Validation : The process of evaluating software at the end of software development to ensure compliance with intended usage

IV&V stands for "independent verification and validation"

Testing Goals Based on Test Process Maturity Level

- Level 0 : There's no difference between testing and debugging
- Level 1 : The purpose of testing is to show correctness
- Level 2 : The purpose of testing is to show that the software doesn't work
- Level 3 : The purpose of testing is not to prove anything specific, but to reduce the risk of using the software
- Level 4 : Testing is a mental discipline that helps all IT professionals develop higher quality software

Level 0 Thinking

- Testing is the same as debugging
- Does not distinguish between incorrect behavior and mistakes in the program
- Does not help develop software that is reliable or safe

This is what we teach undergraduate CS majors

Level 1 Thinking

- Purpose is to show correctness
- Correctness is impossible to achieve
- What do we know if no failures?
  - Good software or bad tests?
- Test engineers have no:
  - Strict goal
  - Real stopping rule
  - Formal test technique
  - Test managers are powerless

This is what hardware engineers often expect

Level 2 Thinking

- Purpose is to show failures
- Looking for failures is a negative activity
- Puts testers and developers into an adversarial relationship
- What if there are no failures?

This describes most software companies.

How can we move to a *team approach* ??

Level 3 Thinking

- Testing can only show the presence of failures
- Whenever we use software, we incur some risk
- Risk may be small and consequences unimportant
- Risk may be great and consequences catastrophic
- Testers and developers cooperate to reduce risk

It describes a few "enlightened" software companies

Level 4 Thinking

A mental discipline that increases quality

- Testing is only one way to increase quality
- Test engineers can become technical leaders of the project
- Primary responsibility is to measure and improve software quality
- Their expertise should help the developers

This is the way "traditional" engineering works

Where Are You?

Are you at level 0, 1, or 2 ?

Is your organization at work at level 0, 1, or 2 ?

Or 3?

We hope to teach you to become "change agents" in your workplace …

Advocates for level 4 thinking

Tactical Goals : Why Each Test ?

If you don't know why you're conducting each test, it won't be very helpful

- Written test objectives and requirements must be documented
- What are your planned coverage levels?
- How much testing is enough?
- Common objective – spend the budget … test until the ship-date …
  - Sometimes called the "date criterion"

Here! Test This!

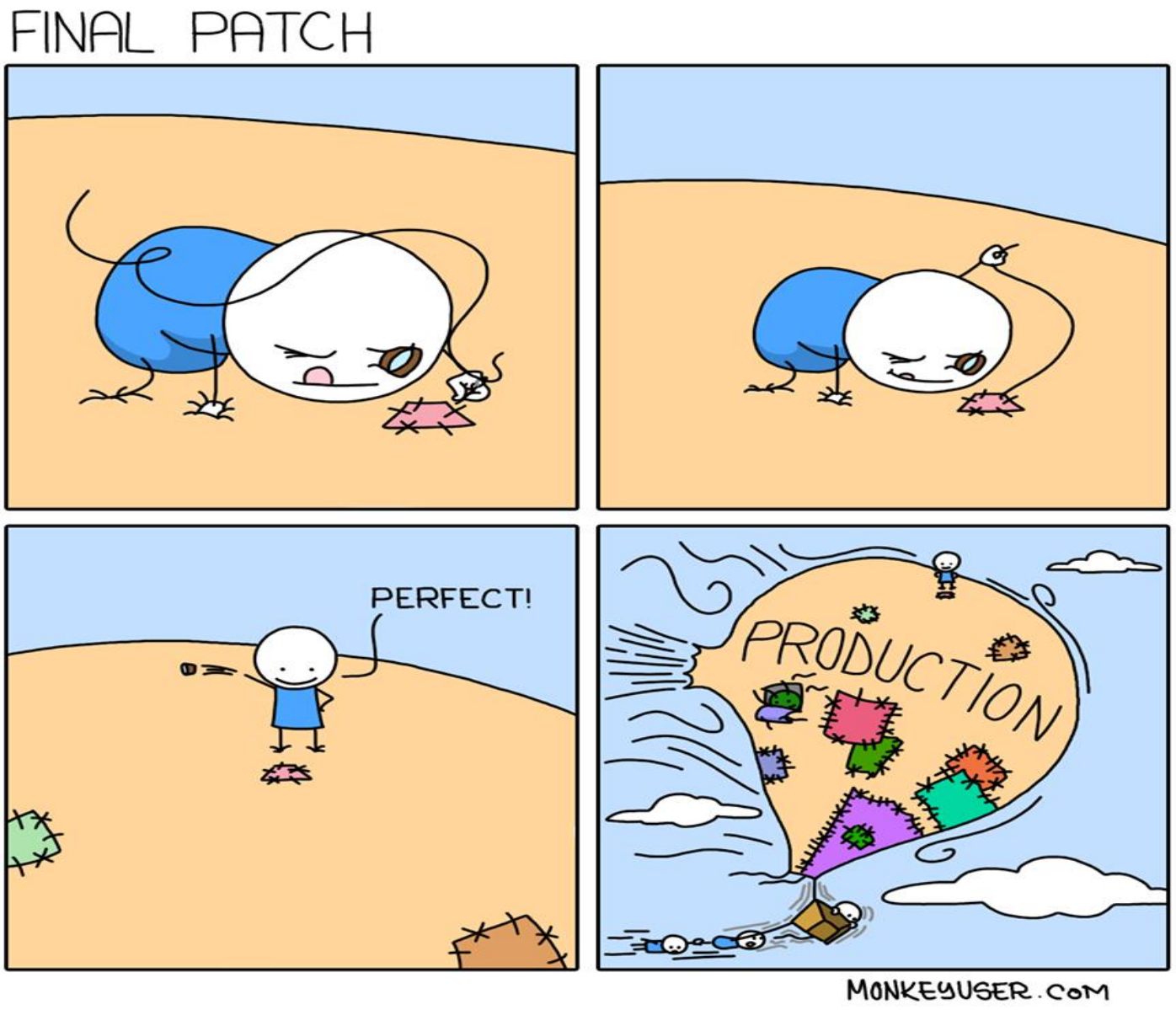
A stack of computer printouts—and no documentation

Why Each Test ?

- 1980: “The software shall be easily maintainable”
- Threshold reliability requirements?
- What fact does each test try to verify?
- Requirements definition teams need testers!

If you don't start planning for each test when the functional requirements are formed, you'll never know why you're conducting the test

Testing



Getting Software Right

- Inspections
- Design discussion
- Statical Analysis
- Testing
- Runtime verification

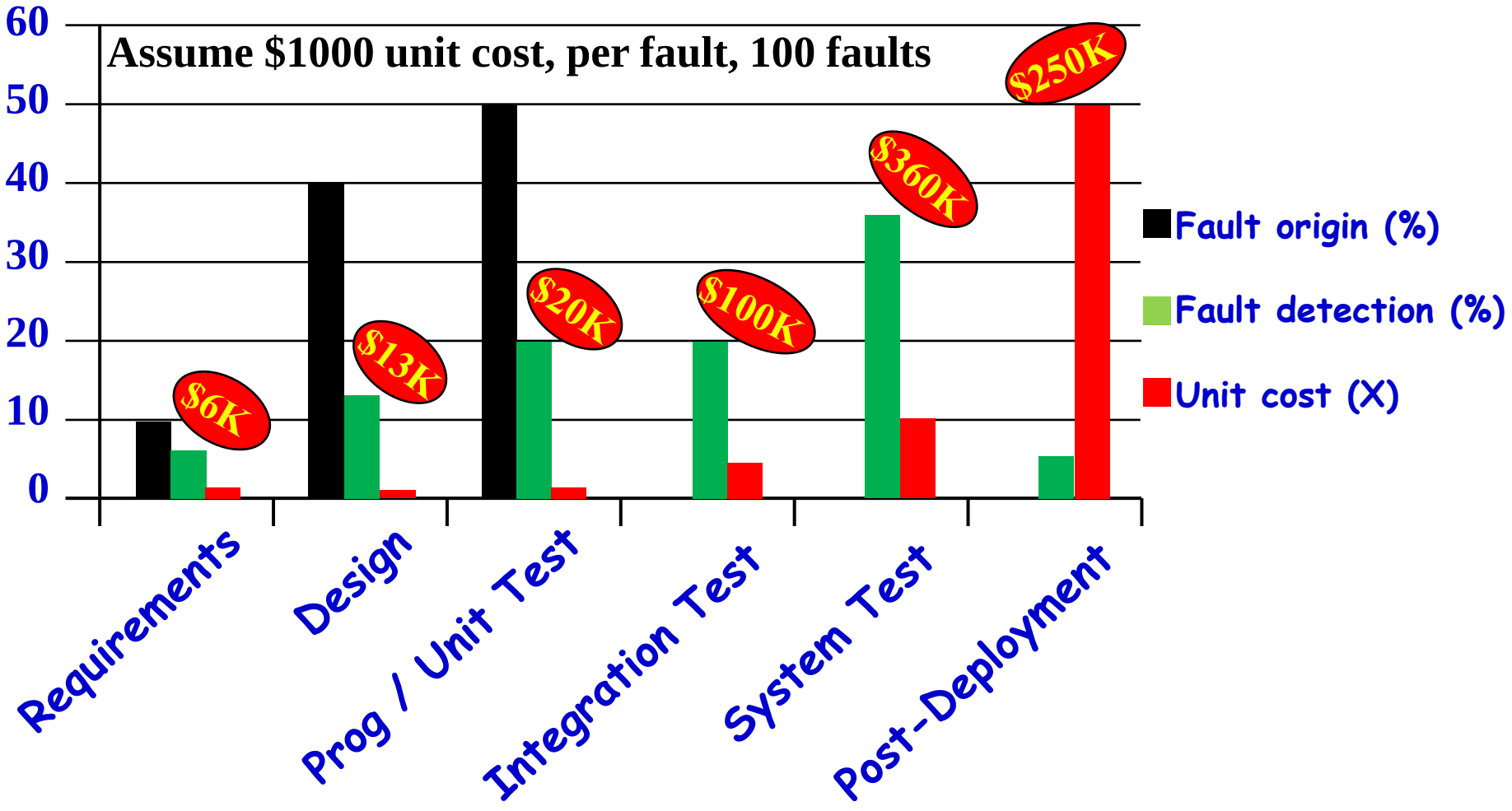
Need to Understand and
Validate software requirement

Cost of Not Testing

- Testing is the most time consuming and expensive part of software development
- Not testing is even more expensive
- If we have too little testing effort early, the cost of testing increases
- Planning for testing after development is prohibitively expensive

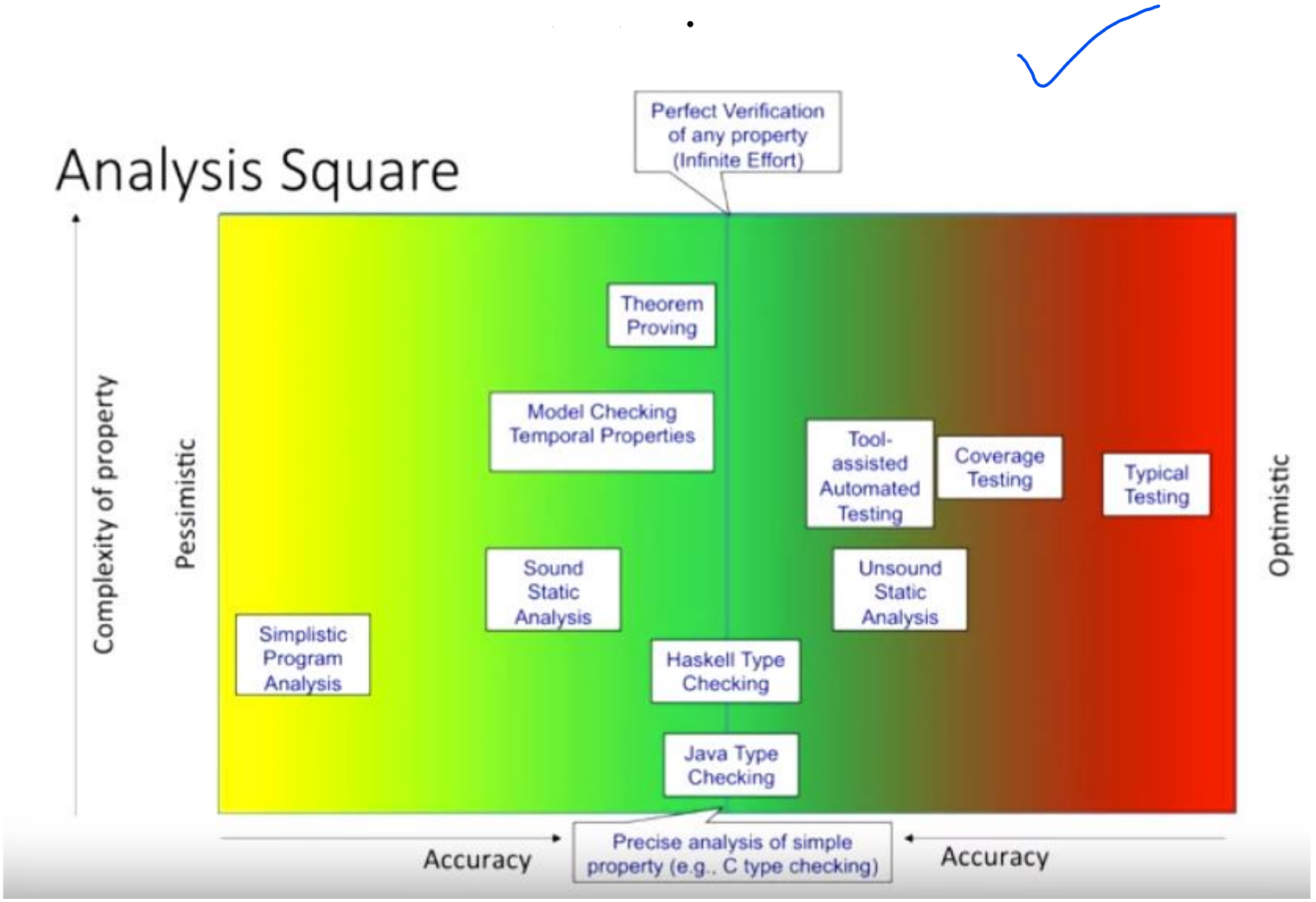
Poor Program Managers might say: "Testing is too expensive."

Cost of Late Testing



Software Engineering Institute; Carnegie Mellon University; Handbook CMU/SEI-96-HB-002

Testing Software Strategies



Complexity of Testing Software

- No other engineering field builds products as complicated as software
- The term correctness has no meaning
  - Is a building correct?
  - Is a car correct?
  - Is a subway system correct?
- Instead of looking for "correctness," wise software engineers try to evaluate software's "behavior" to decide if the behavior is acceptable within consideration of a large number of factors including (but not limited to) reliability, safety, maintainability, security, and efficiency.
- Obviously this is more complex than the naive desire to show the software is correct.

Complexity of Testing Software

- Like other engineers, we must use abstraction to manage complexity
  - This is the purpose of the model-driven test design process
  - The "model" is an abstract structure
- The Model-Driven Test Design (MDTD) process breaks testing into a series of small tasks that simplify test generation.
  - Then test designers isolate their task, and work at a higher level of abstraction by using mathematical engineering structures to design test values independently of the details of software or design artifacts, test automation, and test execution.

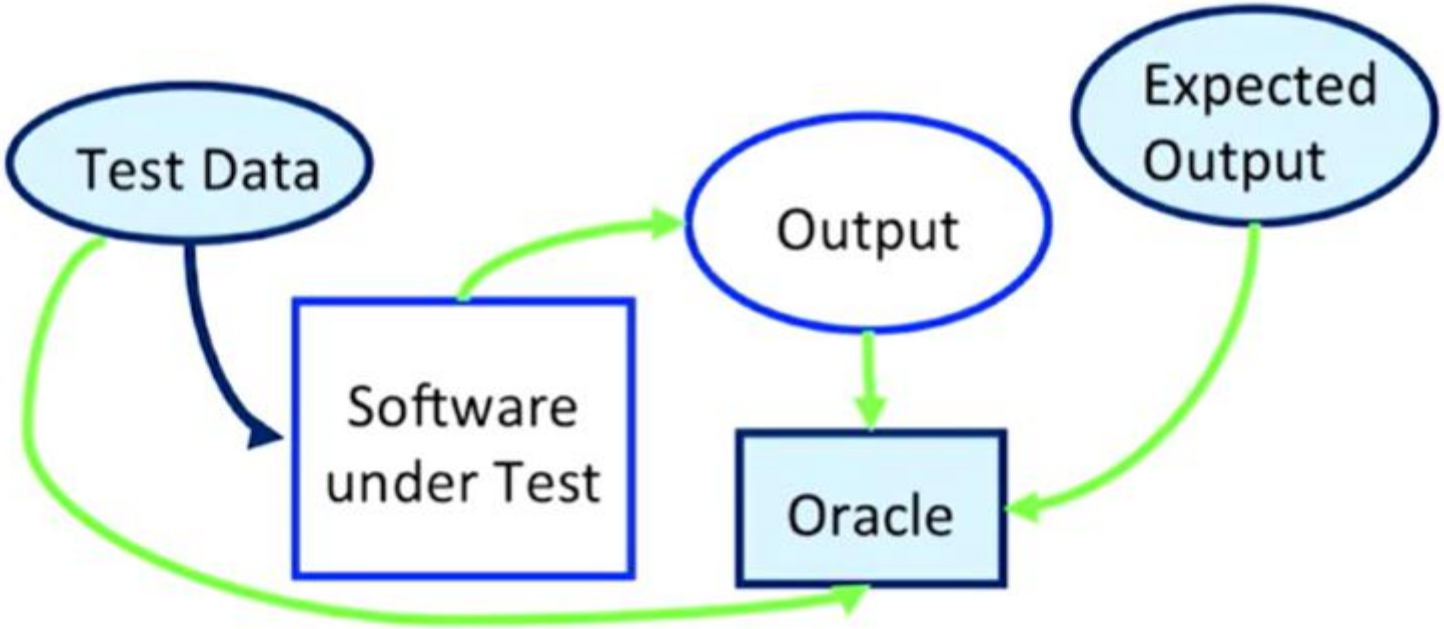
Software Testing Foundations (2.1)

Testing can only show the presence of failures

Not their absence

Software Testing

Test



Anatomy of a Test

Four Components

1. **Setup**
2. **Invocation**
3. **Assessment**
4. **Teardown**

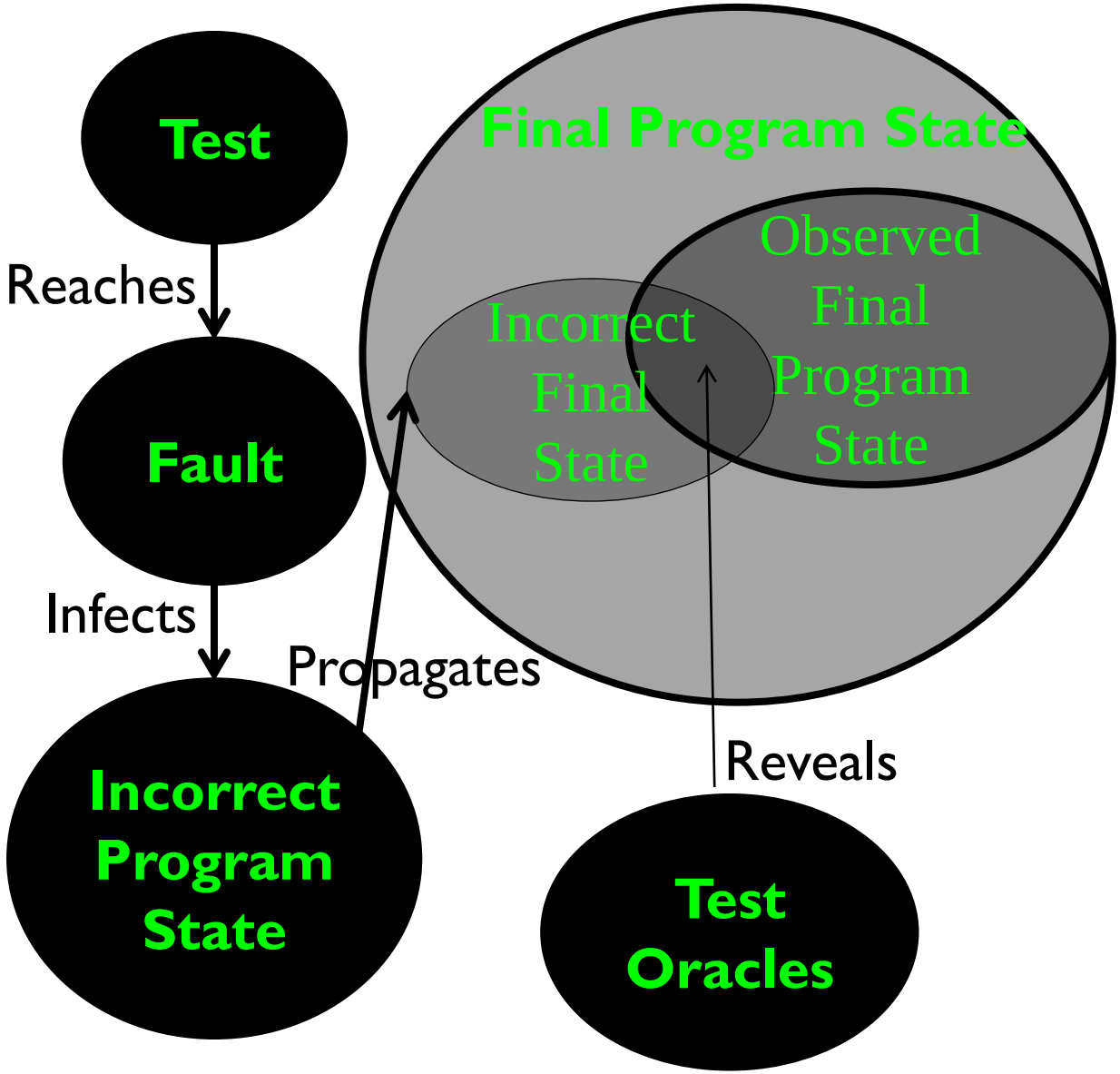
Fault & Failure Model (RIPR)

Four conditions necessary for a failure to be observed

1. **Reachability** : The location or locations in the program that contain the fault must be reached
2. **Infection** : The state of the program must be incorrect
3. **Propagation** : The infected state must cause some incorrect output or final state
4. **Reveal** : The tester must observe part of the incorrect portion of the program state

RIPR Model

- **R**eachability
- **I**nfection
- **P**ropagation
- **R**evealability



Software Testing Activities (2.2)

- Test Engineer : An IT professional who is in charge of one or more technical test activities
  - Designing test inputs
  - Producing test values
  - Running test scripts
  - Analyzing results
  - Reporting results to developers and managers

- Test Manager : In charge of one or more test engineers
  - Sets test policies and processes
  - Interacts with other managers on the project
  - Otherwise supports the engineers

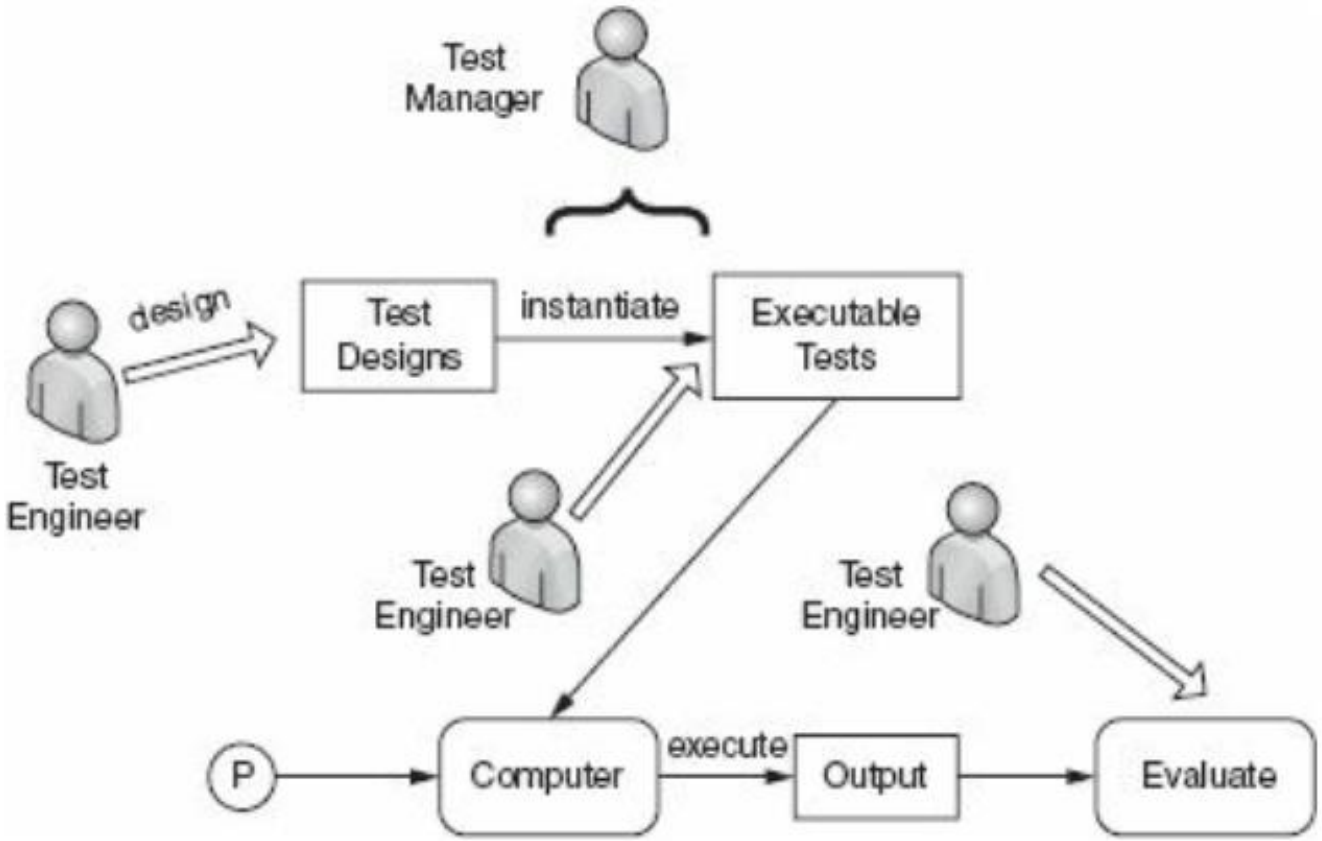


Figure: Activities of test engineer and manager.

Traditional Testing Levels

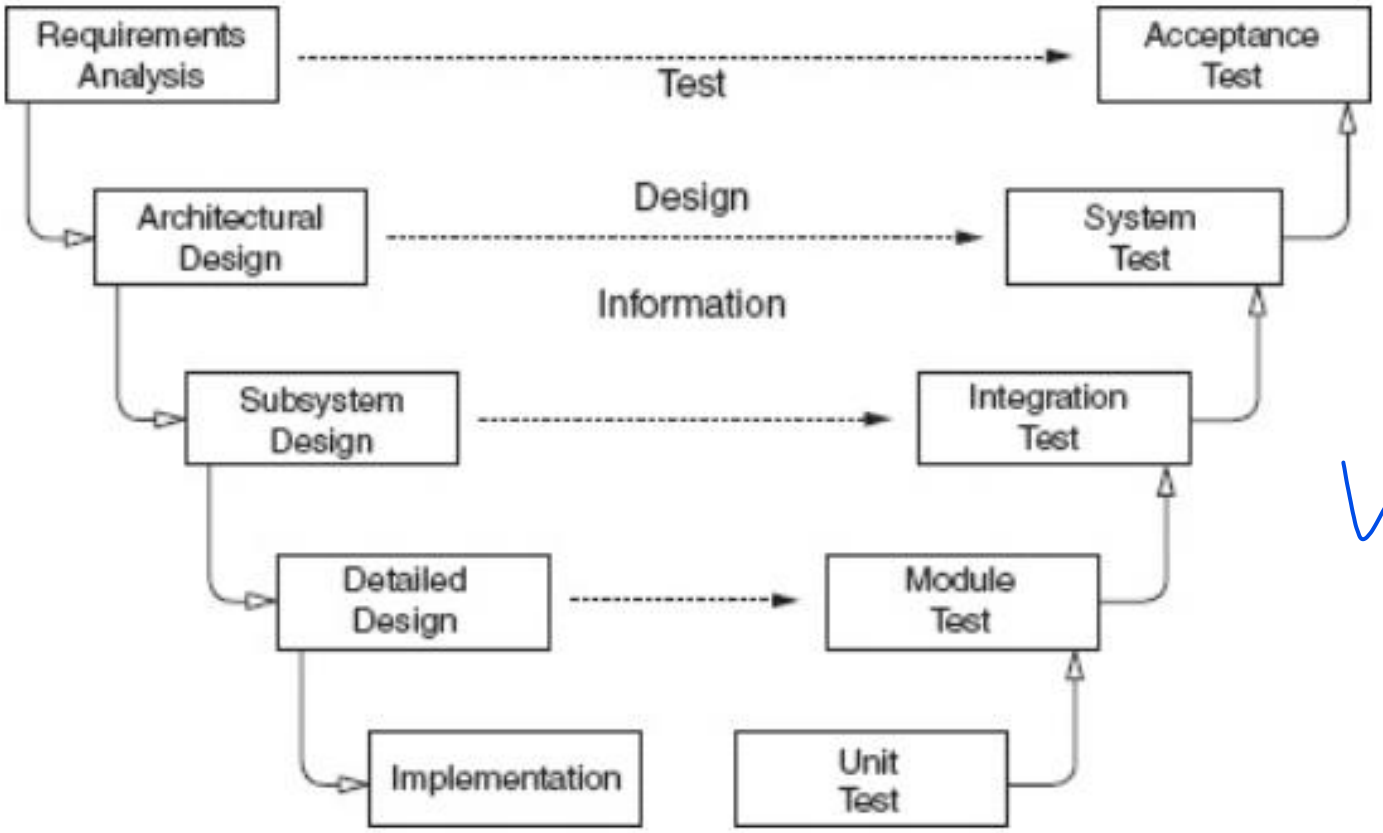


Figure: Software development activities and testing levels – the "V Model"

V-Model

- The *requirements analysis* phase of software development captures the customer's needs.

- *Acceptance testing* is designed to determine whether the completed software in fact meets these needs. In other words, acceptance testing probes whether the software does what the users want.
  - Acceptance testing must involve users or other individuals who have strong domain knowledge.

V-Model Cont.

- The *architectural design* phase of software development chooses *components and connectors* that together realize a system whose specification is intended to meet the previously identified requirements.
- *System testing* is designed to determine whether the assembled system meets its specifications. It assumes that the pieces work individually, and asks if the system works as a whole.
  - This level of testing usually looks for design and specification problems.
  - It is a very expensive place to find lower-level faults and is usually not done by the programmers, but by a separate testing team

V-Model Cont.

- The *subsystem design* phase of software development specifies the structure and behavior of **subsystems**, each of which is intended to satisfy some function in the overall architecture. Often, the subsystems are adaptations of previously developed software.
- *Integration testing* is designed to assess whether the interfaces between modules (defined below) in a subsystem have consistent assumptions and communicate correctly.
  - Integration testing must assume that modules work correctly.
  - Integration testing is usually the responsibility of members of the development team

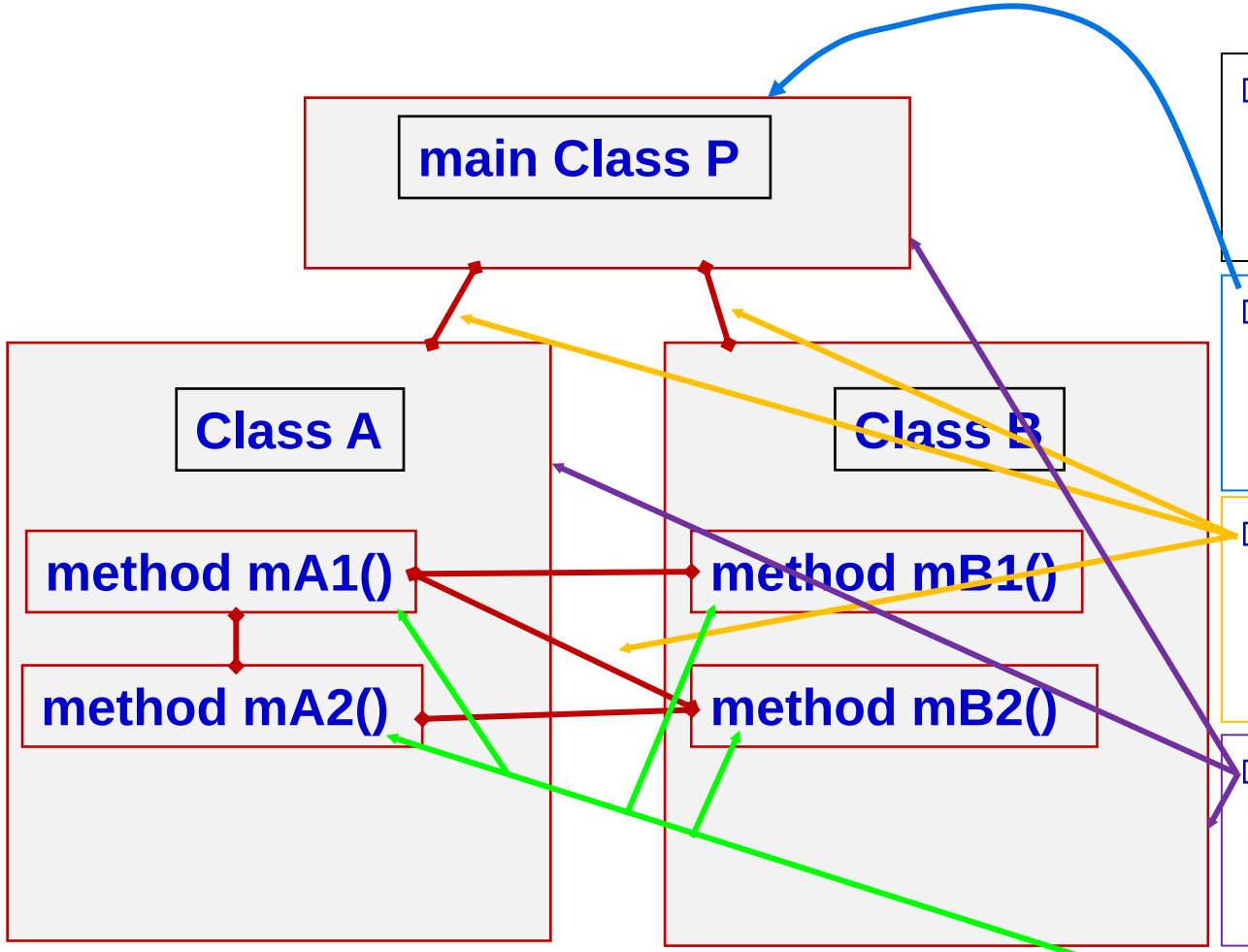
V-Model Cont.

- The *detailed design* phase of software development determines the structure and behavior of **individual modules**. A *module* is a collection of related units that are assembled in a file, package, or class.
  - This corresponds to a file in C, a package in Ada, and a class in C++ and Java.
- *Module testing* is designed to assess individual modules in isolation, including how the component units interact with each other and their associated data structures.
  - Most software development organizations make module testing the responsibility of the programmer; hence the common term *developer testing*.

V-Model Cont.

- *Implementation* is the phase of software development that actually produces code. A program *unit*, or procedure, is one or more contiguous program statements, with a name that other parts of the software use to call it.
  - Units are called functions in C and C++, procedures or functions in Ada, methods in Java, and subroutines in Fortran.
- *Unit testing* is designed to assess the units produced by the implementation phase and is the "lowest" level of testing.
  - As with module testing, most software development organizations make unit testing the responsibility of the programmer, again, often called developer testing.

Traditional Testing Levels

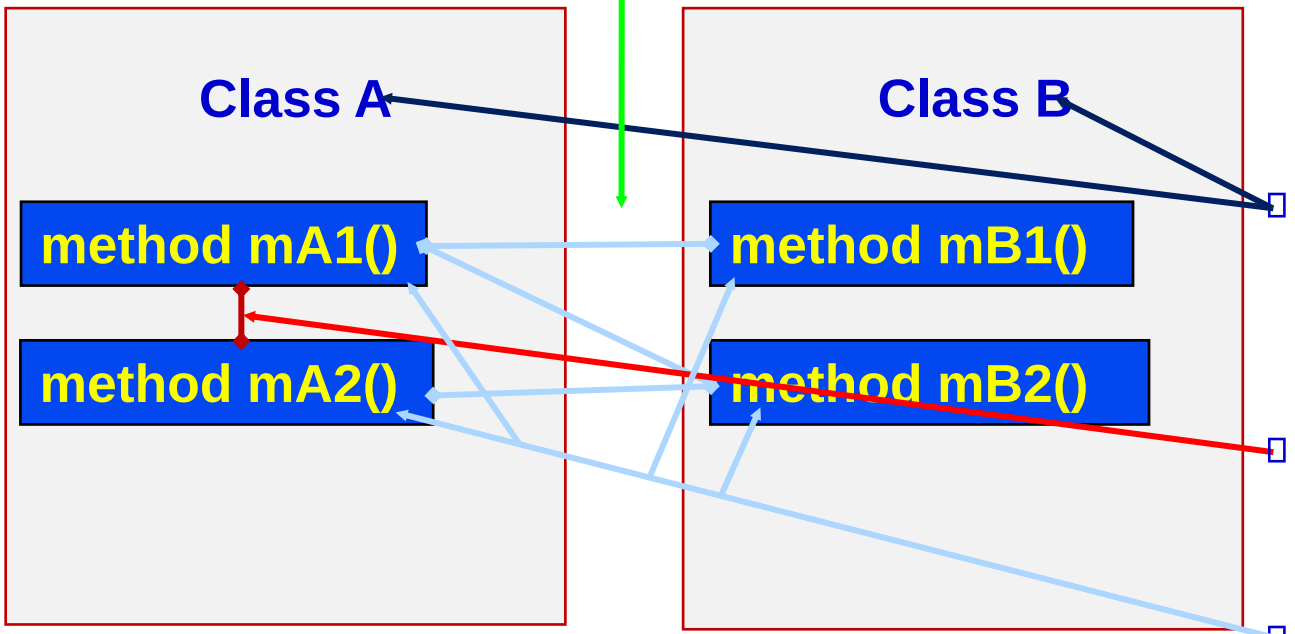


- **Acceptance testing** : Is the software acceptable to the user?
- **System testing** :Test the overall functionality of the system
- **Integration testing** : Test how modules interact with each other
- **Module testing (developer testing)** : Test each class, file, module, component
- **Unit testing (developer testing)** : Test each unit (method) individually

This view obscures underlying similarities

Object-Oriented Testing Levels

- **Inter-class testing** :
  Test multiple classes together



- **Intra-class testing** :
  Test an entire class as sequences of calls
- **Inter-method testing** :
  Test pairs of methods in the same class
- **Intra-method testing** :
  Test each method individually

Summary: Why Do We Test Software ?

A tester's goal is to eliminate faults as early as possible

- **Improve quality**
- **Reduce cost**
- **Preserve customer satisfaction**